

Master Spark Architecture — Version 1

I have treated the uploaded text as the specification for the architecture you want to build. Your core objective is not memorizing Spark terminology; it is being able to move **backward** and forward between code, plans, execution, metrics, and bottlenecks.

I am **not** creating the PDF yet. This is the architecture for you to inspect and challenge first, as requested.

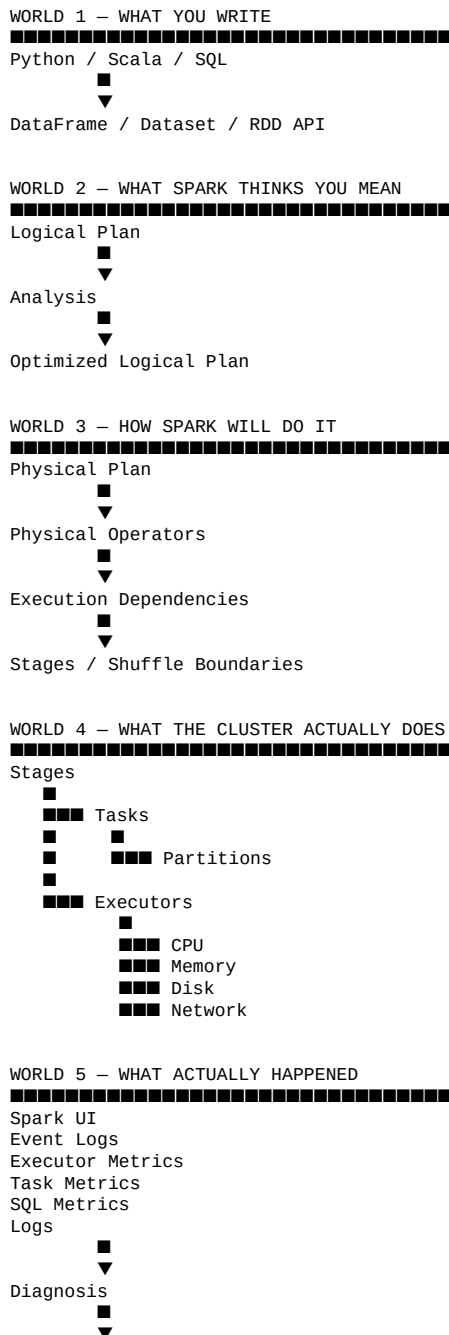
I also corrected several relationships in your proposed sequence where the simplified version would create the wrong mental model.

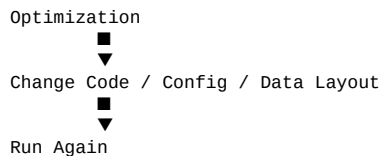
1. Executive Mental Model

The most important idea is this:

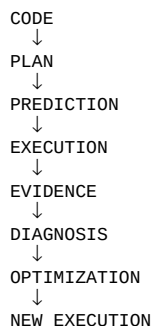
> **Spark is a system that takes a high-level description of computation, turns it into an executable distributed plan, runs that plan over partitions of data, and exposes enough evidence for you to determine whether reality matched the plan.**

Think of Spark as **five connected worlds**:





That gives us the fundamental feedback loop:

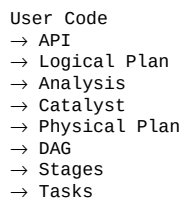


Your requested **READ → PREDICT → VERIFY** method fits directly into this architecture.

2. The First Major Correction: Spark Does NOT Have One Single Pipeline

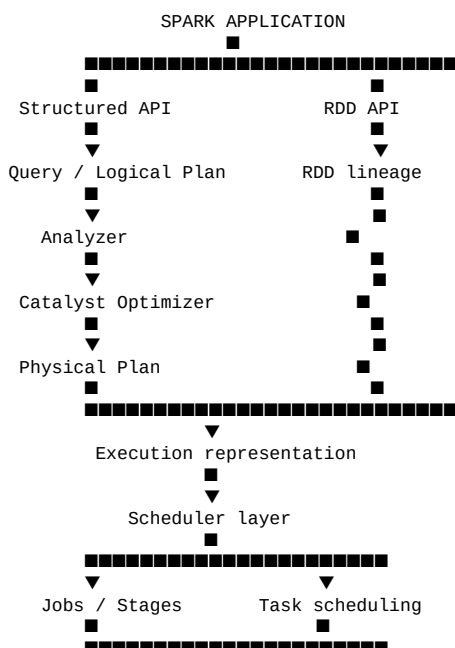
Your proposed chain is useful, but there is an important correction.

You proposed approximately:



That is a good **SQL/DataFrame mental model**, but it is not universal Spark architecture.

The better model is:



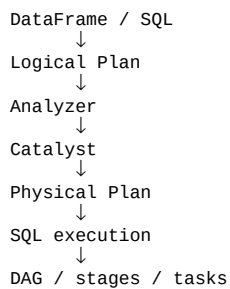


This distinction matters because **Catalyst belongs primarily to Spark SQL/DataFrame/Dataset planning**, not to every Spark operation.

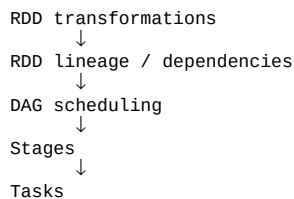
Spark's SQL/DataFrame APIs provide structural information that Spark can exploit for optimization; the execution engine is then shared underneath those APIs.

So our master architecture must have **two branches that converge at execution**:

Structured branch



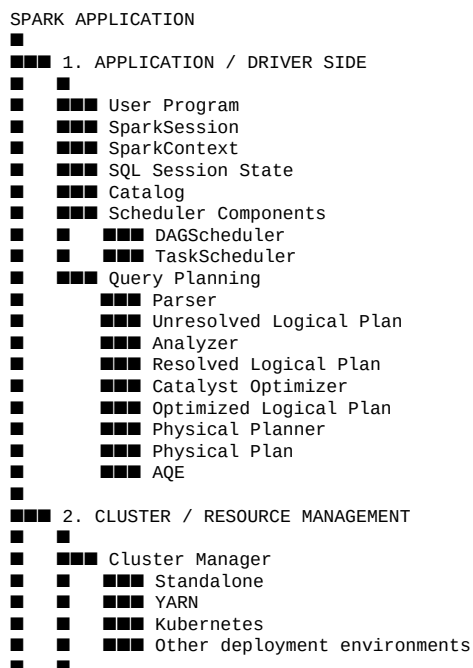
RDD branch



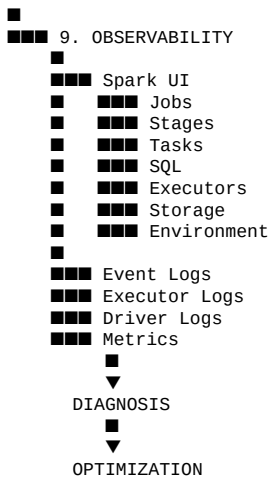
This is one of the first misconceptions we should eliminate.

3. Giant Architecture Map

Here is the architecture I recommend making our **stable master map**.



- ■■■ Resource Allocation
 - ■■■ Executors
 - ■■■ Executor Cores
 - ■■■ Executor Memory
 - ■■■ Memory Overhead
- 3. DATA SOURCE / STORAGE
 - ■■■ Object Storage / HDFS / Local FS
 - ■■■ Files
 - ■■■ File Metadata
 - ■■■ File Splitting
 - ■■■ Input Partitions
 - ■■■ Parquet / ORC / JSON / etc.
 - ■■■ Partition Pruning
 - ■■■ Predicate Pushdown
 - ■■■ Column Pruning
- 4. QUERY PLANNING
 - ■■■ User Expression
 - ■■■ Logical Plan
 - ■■■ Analysis
 - ■■■ Catalog Resolution
 - ■■■ Type Resolution
 - ■■■ Function Resolution
 - ■■■ Logical Optimization
 - ■■■ Statistics
 - ■■■ CBO where applicable
 - ■■■ Optimized Logical Plan
 - ■■■ Physical Planning
 - ■■■ Physical Operators
- 5. EXECUTION
 - ■■■ Action
 - ■■■ Job
 - ■■■ Stage
 - ■■■ Task
 - ■■■ Partition
 - ■■■ Task execution
 - ■■■ Shuffle boundary
 - ■■■ Runtime
 - ■■■ CPU
 - ■■■ Memory
 - ■■■ Disk
 - ■■■ Network
- 6. DATA MOVEMENT
 - ■■■ Narrow Dependency
 - ■■■ Wide Dependency
 - ■■■ Shuffle
 - ■■■ Map-side processing
 - ■■■ Shuffle Write
 - ■■■ Shuffle Data
 - ■■■ Shuffle Metadata
 - ■■■ Shuffle Fetch
 - ■■■ Reduce-side processing
 - ■■■ Spill
 - ■■■ Memory → Disk
 - ■■■ Merge / Read
- 7. MEMORY
 - ■■■ JVM / Executor Memory
 - ■■■ Unified Memory
 - ■■■ Execution Memory
 - ■■■ Storage Memory
 - ■■■ User / Other Memory
 - ■■■ Memory Overhead
 - ■■■ Off-heap where applicable
 - ■■■ Cache / Persist
 - ■■■ Aggregation
 - ■■■ Join Structures
 - ■■■ Shuffle Structures
 - ■■■ Spill
 - ■■■ GC
- 8. ADAPTIVE EXECUTION
 - ■■■ Initial Physical Plan
 - ■■■ Runtime Statistics
 - ■■■ Query Stage Completion
 - ■■■ Re-optimization
 - ■■■ Partition Coalescing
 - ■■■ Skew Handling
 - ■■■ Join Adaptation
 - ■■■ Continue Execution

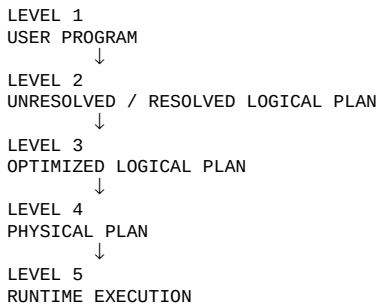


This is the architecture I recommend we use going forward.

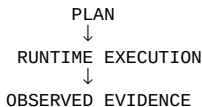
4. The Most Important Mental Model: Five Representations

Your requested five-level model is correct and extremely useful.

But I would make it slightly more precise:



And there is a critical observation layer:



What each means

- | LevelSpark is asking |
- | ----- |
- | User code | **What did the developer request?** |
- | Logical plan | **What computation does that request mean?** |
- | Optimized logical plan | **Can I represent that computation more efficiently?** |
- | Physical plan | **Which algorithms/operators should execute it?** |
- | Runtime | **How do I actually execute those operators over distributed partitions?** |
- | UI/metrics | **What actually happened?** |

The distinction between **planned** and **actual** execution is fundamental. Your specification correctly makes this mandatory.

5. One Query Through the Entire System

We'll use your requested example, with one necessary correction: give the aggregate column an alias.

```
result = (
    df1
    .join(df2, "customer_id")
    .groupBy("country")
    .agg(sum("sales").alias("sales"))
    .filter("sales > 100000")
)

result.write.parquet("/output")
```

The important thing:

Nothing substantial is executed merely because you wrote these transformations.

The transformations describe computation.

The write is an **action** that causes execution.

Conceptually:

```
Python code
↓
PySpark API
↓
Spark SQL / JVM-side query representation
↓
Logical plan
↓
Analysis
↓
Optimized logical plan
↓
Physical plan
↓
Execution
↓
Jobs
↓
Stages
↓
Tasks
↓
Partitions
↓
Executors
↓
Output
```

6. What Happens to the Query

Step 1 — Python

You write:

```
df1.join(...)
```

PySpark is the user-facing language layer.

At this point you are describing computation.

You are **not manually specifying**:

```
Executor 4
read file X
partition data Y
shuffle these records
run task Z
```

You are expressing intent.

7. Logical Plan

Conceptually Spark sees something like:

```
Write
|
Filter
  sales > 100000
|
Aggregate
  group by country
  sum(sales)
|
Join
  customer_id
 / \
df1  df2
```

This is not yet:

```
SortMergeJoin
HashAggregate
Exchange
Task
Executor
```

Those are later decisions.

That distinction is crucial.

8. Unresolved Logical Plan

Initially Spark may have references that still need resolution:

```
Join
  ■■■ customer_id
  ?
```

Spark still needs to determine things such as:

- Which `customer\_id`?
- Does the column exist?
- What is its data type?
- Which relation does it belong to?
- What does `sum` resolve to?
- What is the schema?
- What catalog/table metadata applies?

So:

```
UNRESOLVED

"customer_id"
  ↓
"some specific attribute from df1/df2"
```

9. Analyzer

The Analyzer resolves the logical plan.

Conceptually:

```
Unresolved Attribute
  ↓
Catalog / Schema / Session information
  ↓
Resolved Attribute
```

It resolves:

- relations
- columns
- functions
- types
- aliases
- expressions
- references

Then we have:

```
RESOLVED LOGICAL PLAN
```

Now Spark knows what the query means.

But it still hasn't necessarily decided **how to execute it**.

10. Catalyst Optimization

Now optimization rules can transform the logical representation.

For example:

```
BEFORE

Filter
|
Aggregate
|
Join
```

A filter may sometimes be pushed earlier if semantics allow:

```
Filter
  ↓
smaller input
  ↓
Join
```

This can reduce:

```
rows
  ↓
CPU
  ↓
network
  ↓
shuffle
  ↓
memory
```

Another example:

```
SELECT customer_id, country, sales
```

instead of carrying:

```
customer_id
country
sales
address
phone
email
...
```

Spark can prune unnecessary columns.

So:

```
Logical Plan
  ↓
Optimization Rules
  ↓
Optimized Logical Plan
```

Catalyst contains analyzer, optimizer, planning and adaptive extension points in the Spark SQL architecture; exact internal rule sets are version-dependent.

11. What Catalyst Does NOT Know

This is important.

Catalyst does **not** simply possess omniscient knowledge of your cluster.

There is a difference between:

```
STATIC QUERY INFORMATION
```

and:

```
RUNTIME INFORMATION
```

Before execution, Spark may know:

- schema
- expressions
- statistics when available
- configuration
- estimated sizes
- available logical transformations

But runtime may reveal:

```
actual rows
actual partition sizes
actual shuffle sizes
actual skew
actual task durations
actual runtime statistics
```

This is one reason AQE exists.

12. Physical Planning

Now Spark must answer:

> **How should this computation actually be executed?**

For the join it might choose:

BroadcastHashJoin

or:

SortMergeJoin

or another applicable strategy.

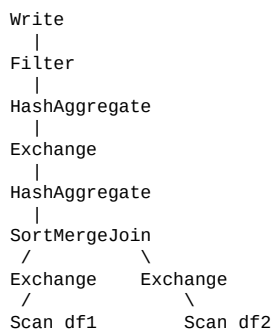
For aggregation it may use:

HashAggregate

or:

SortAggregate

The physical plan could conceptually become:



Now we have something much closer to executable behavior.

13. The Exchange Is a Major Architectural Signal

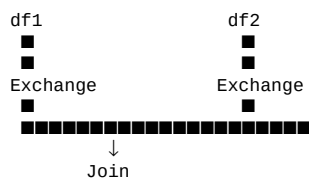
Whenever you see:

Exchange

you should immediately ask:

> **Why must data move between partitions?**

For example:



That generally indicates redistribution of data.

That redistribution is associated with a **shuffle boundary** in the execution architecture.

This is where:

CPU
Memory
Disk

Network

start interacting heavily.

14. Physical Plan $\neq$ DAG

This is another correction to lock into the master model.

Do **not** think:

```
Physical Plan
      =
DAG
```

They are related but represent different things.

Physical plan

Answers:

> **Which execution operators should perform the computation?**

Example:

```
Scan
Exchange
Sort
SortMergeJoin
HashAggregate
```

DAG / execution graph

Answers more directly:

> **How is the computation divided by dependencies and scheduling boundaries?**

For example:

```
Stage 0
  ↓
Shuffle
  ↓
Stage 1
  ↓
Shuffle
  ↓
Stage 2
```

The DAG scheduler operates on the execution dependencies and divides work into stages around shuffle boundaries.

So our mental chain is:

```
Physical Plan
  ↓
Execution representation / dependencies
  ↓
Shuffle boundaries
  ↓
Stages
  ↓
Tasks
```

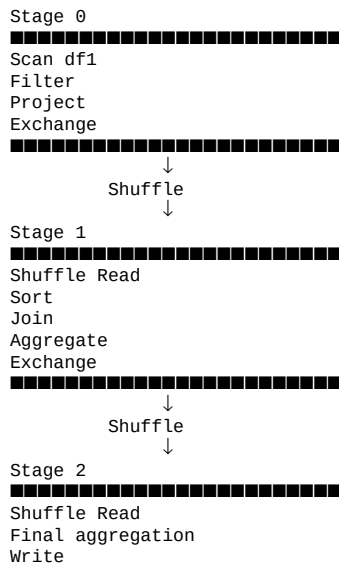
Not:

Physical Plan magically becomes a picture called DAG.

15. Stage

A stage is essentially a set of tasks that can execute together against a partitioned dataset without crossing another shuffle boundary inside that stage.

Example:



Therefore:

> **A stage is not a transformation.**

One stage can contain multiple operators.

Your specification explicitly calls out this distinction, and it should remain central to the model.

16. Task

Now we reach the physical execution unit.

Suppose:

```
Stage 1
↓
200 partitions
```

Conceptually:

```
Partition 0 → Task 0
Partition 1 → Task 1
Partition 2 → Task 2
...
Partition 199 → Task 199
```

Therefore:

```
1 partition
↓
1 task for that stage
```

But:

```
partition ≠ task
```

The partition is the **data subdivision**.

The task is the **attempt to process that subdivision for a stage**.

That distinction is critical.

17. Executor

Suppose:

```
10 executors
4 cores each
```

Simplified maximum concurrent task capacity:

```
10 × 4
= 40 task slots
```

If a stage has:

```
200 tasks
```

then the simplified picture is:

```
200 tasks
÷
40 concurrent slots
≈
5 waves
```

But this is **not a runtime prediction**.

Real execution is affected by:

- task duration
- scheduling
- locality
- skew
- executor availability
- failed tasks
- speculative execution
- dynamic allocation
- resource contention

Exactly as your specification requires, we will treat these calculations as simplified capacity models rather than promises about runtime.

18. Partition Architecture

We need to stop using "partition" as though it means one thing.

There are multiple concepts:

Concept	Meaning
File	Physical storage object
File split/input partition	Portion of input data assigned for reading
Spark partition	Logical/runtime subdivision processed by tasks

| Shuffle partition | Partition produced by a shuffle exchange |

| Task | Execution attempt for a partition in a stage |

| Executor | JVM/process/container executing tasks |

| Executor core | Resource allowing task concurrency |

The relationship is approximately:

```

SOURCE FILES
  ↓
INPUT PARTITIONS
  ↓
TASKS
  ↓
EXECUTORS

```

After a shuffle:

```

MAP-SIDE PARTITIONS
  ↓
SHUFFLE
  ↓
SHUFFLE PARTITIONS
  ↓
REDUCE-SIDE TASKS

```

19. Data Movement Architecture

This should become one of our most important performance maps.

```

Storage
  ↓
File
  ↓
Input split
  ↓
Spark input partition
  ↓
Task
  ↓
Executor
  ↓
Transformation
  ↓
Narrow dependency
  ■
  ■ no redistribution required
  ↓
Next partition

```

Or, when redistribution is required:

```

Executor
  ↓
Map-side processing
  ↓
Shuffle Write
  ↓
Shuffle Data
  ↓
Disk / local storage
  ↓
Network
  ↓
Shuffle Fetch
  ↓
Reduce-side Task
  ↓
Executor
  ↓
Join / Aggregate

```

This is where performance engineering becomes much more concrete.

20. CPU / Memory / Disk / Network

For every operation we should eventually ask four questions.

CPU

What computation must happen?

Examples:

```
hashing
sorting
aggregation
serialization
deserialization
expression evaluation
```

Memory

What must be held in memory?

Examples:

```
hash maps
join build side
aggregation state
shuffle buffers
cached data
```

Disk

What happens when memory isn't sufficient?

Potentially:

```
spill
shuffle files
temporary files
```

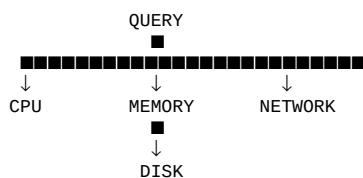
Network

What data must cross executor boundaries?

Especially:

```
shuffle
broadcast
remote block fetch
```

This gives us the performance model:



And the bottleneck isn't always where beginners initially look.

21. Shuffle Architecture

The complete model:

```

WHY REDISTRIBUTION IS REQUIRED
↓
Partitioning Requirement
↓
Map-side Processing
↓
Shuffle Write
↓
Shuffle Data / Metadata
↓
Shuffle Storage
↓
Network Fetch where needed
↓
Shuffle Read
↓
Reduce-side Processing
↓
Join / Aggregate / Sort / etc.

```

A shuffle can create pressure in several places simultaneously:

```

CPU
  ■■■ hashing / sorting / serialization

MEMORY
  ■■■ aggregation / buffers / join state

DISK
  ■■■ spill / shuffle files

NETWORK
  ■■■ shuffle transfer

SCHEDULER
  ■■■ many tasks / stragglers

```

Therefore:

> *****"Shuffle is slow" is not a diagnosis.****

We must determine ****which part of the shuffle is expensive and why****.

22. Join Decision Architecture

The join decision should not be memorized as:

> Small table → broadcast.

That is too simplistic.

The actual reasoning needs to consider:

```

JOIN
  ■
  ■■■ Can one side be broadcast?
  ■
  ■■■ Yes → Broadcast Hash Join candidate
  ■
  ■■■ No
  ■
  ■■■ Is a sort-merge strategy applicable?
  ■
  ■■■ SortMergeJoin candidate
  ■
  ■■■ Is shuffled hash join applicable?
  ■
  ■■■ Other join strategies

```

Then evaluate:

```

Statistics
+
Configuration
+
Join type
+

```

Data sizes
+
Memory
+
Hints
+
Runtime information

AQE can change certain join decisions during execution. For example, Spark's documented AQE behavior includes converting a sort-merge join to a broadcast hash join when runtime statistics show that a side is sufficiently small.

So we must separate:

INITIAL JOIN DECISION

from:

RUNTIME JOIN ADAPTATION

23. AQE — Where It Fits

AQE belongs ****between planned execution and continued runtime execution****, not as a completely separate planning universe.

Conceptually:

```
Optimized Logical Plan
  ↓
Initial Physical Plan
  ↓
Execution begins
  ↓
Runtime statistics become available
  ↓
AQE
  ↓
Re-optimize portions of execution
  ↓
Continue
```

AQE can address things such as:

post-shuffle partition coalescing
skew handling
join strategy adaptation

and related adaptive optimizations.

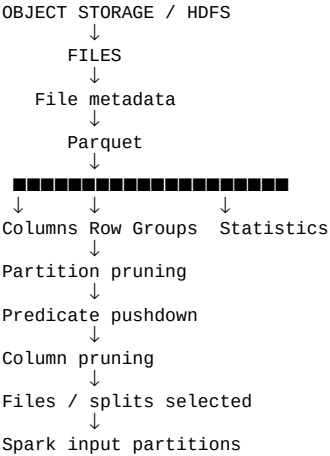
Apache Spark documentation states that AQE has been enabled by default since Spark 3.2.0.

This is an important boundary:

```
BEFORE EXECUTION
  ↓
Spark estimates / plans
  ↓
EXECUTION
  ↓
ACTUAL STATISTICS
  ↓
AQE
  ↓
RUNTIME ADAPTATION
```

24. Storage Architecture

The storage model needs to remain separate from Spark execution.



Important distinction:

Storage partition

Example:

/year=2026/month=09/day=21/

is a ****data layout concept****.

Spark partition

The unit Spark processes as distributed work.

They are related, but they are ****not the same object****.

This distinction will become extremely important when we later study:

- partition pruning
- repartition()
- coalesce()
- shuffle partitions
- file sizing
- small files

...

25. Small Files

Suppose you have:

10,000,000 files

even if their total size isn't enormous.

The problem isn't necessarily just data volume.

You can incur overhead from:

```
file listing
metadata handling
file opening
task creation
scheduling
small task execution
```

So:

10 TB in well-sized files

and:

10 TB in millions of tiny files

can produce very different execution behavior.

This is why storage architecture belongs inside Spark performance architecture.

26. Memory Architecture

The simplified executor model we will use initially:

```
EXECUTOR
■
■■■ JVM / Process
■
■■■ Execution-related memory
■   ■■■ Shuffle
■   ■■■ Join
■   ■■■ Aggregation
■   ■■■ Sort
■
■■■ Storage-related memory
■   ■■■ Cache / Persist
■
■■■ Other / user memory
■
■■■ Memory overhead
■
■■■ Off-heap where configured / applicable
```

When memory pressure occurs, the outcome isn't automatically:

OOM

There are several possible paths:

```
Memory pressure
■
■■■ Spill
■   ↓
■   Disk
■
■■■ GC pressure
■
■■■ Reduced cache retention
■
■■■ Memory overhead exhaustion
■
■■■ OOM
```

And there are fundamentally different OOM locations:

```
Driver OOM
≠
Executor OOM
≠
Container / memory-overhead failure
```

We will preserve those distinctions.

27. Performance Architecture

The master performance tree should be:

```
SLOW JOB
■
■■■ 1. Is the query doing too much work?
■   ■■■ Poor filtering
```

```

■ ■ ■ Poor column pruning
■ ■ ■ Excessive data scanned
■
■ ■ ■ 2. Is data movement excessive?
■ ■ ■ Shuffle volume
■ ■ ■ Broadcast
■ ■ ■ Network
■
■ ■ ■ 3. Is data unevenly distributed?
■ ■ ■ Skew
■ ■ ■ Partition imbalance
■
■ ■ ■ 4. Is CPU the bottleneck?
■ ■ ■ Sorting
■ ■ ■ Hashing
■ ■ ■ Serialization
■ ■ ■ Complex expressions
■
■ ■ ■ 5. Is memory the bottleneck?
■ ■ ■ Aggregation
■ ■ ■ Join
■ ■ ■ Cache
■ ■ ■ GC
■
■ ■ ■ 6. Is disk the bottleneck?
■ ■ ■ Spill
■ ■ ■ Shuffle I/O
■
■ ■ ■ 7. Is scheduling the bottleneck?
■ ■ ■ Too many tiny tasks
■ ■ ■ Scheduler delay
■ ■ ■ Resource starvation
■
■ ■ ■ 8. Is the cluster itself the bottleneck?
■ ■ ■ Insufficient executors
■ ■ ■ Insufficient cores
■ ■ ■ Insufficient memory
■ ■ ■ Resource contention

```

But the real diagnostic method is:

```

SYMPTOM
↓
EVIDENCE
↓
METRIC
↓
HYPOTHESIS
↓
CONTROLLED TEST
↓
VALIDATION
↓
FIX
↓
TRADE-OFF

```

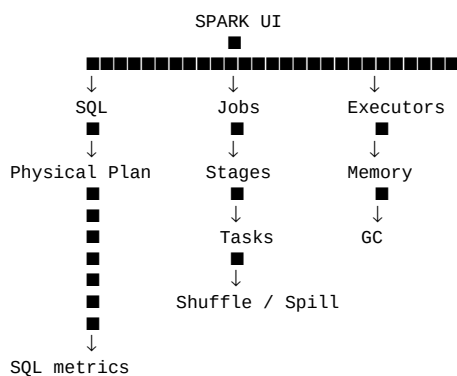
This directly follows the diagnostic structure you requested.

■■■

28. Spark UI = Reality Check

The UI isn't simply a monitoring dashboard.

For us it becomes an ****execution evidence system****.



Spark's current UI documentation exposes task duration, GC time, scheduler delay, peak execution memory, shuffle-related information, and SQL execution details; SQL stages can also be cross-referenced with WholeStageCodegen and SQL plan information.

So eventually I want you to be able to see:

```
Stage 7
■
■■■ 199 tasks → 30 sec
■■■ 1 task → 18 min
```

and immediately think:

```
task imbalance
↓
partition imbalance
↓
possible skew
↓
inspect shuffle read
↓
identify operator
↓
identify join/grouping key
↓
return to source code
```

That is the production-engineering skill you are aiming for.

29. Planned vs Actual

This distinction deserves its own permanent mental model.

EXPLAIN

What Spark intends to do.

UI

What actually happened.

Example:

```
PHYSICAL PLAN
Exchange
↓
SortMergeJoin
```

Prediction:

```
Shuffle required
Sorting required
Join execution required
```

Actual UI might reveal:

```
Shuffle Write = 800 GB
Shuffle Read = 800 GB
Task 0 = 20 sec
Task 127 = 24 min
Spill = 500 GB
GC = 35%
```

Now our diagnosis becomes:

```
PLAN
↓
SortMergeJoin
↓
```

Shuffle
↓
ACTUAL
↓
One giant partition
↓
Skew
↓
Huge spill
↓
Straggler

The plan told you **what should happen**.

The UI tells you **what actually happened**.

You need both.

30. Reverse Debugging

This is probably the most valuable skill in the entire architecture.

Start here:

SPARK UI

and work backward:

Symptom
↓
Metric
↓
Task
↓
Partition
↓
Stage
↓
Physical Operator
↓
Exchange / Dependency
↓
Logical Operation
↓
User Code

Example:

ONE TASK = 18 MINUTES

↓

Why is this task larger?

↓

Huge Shuffle Read

↓

Why is this partition huge?

↓

Key distribution is skewed

↓

Which operation created the shuffle?

↓

SortMergeJoin

↓

Which join key?

↓

customer_id

↓

Original code:
df1.join(df2, "customer_id")

Now you've diagnosed the problem without guessing.

31. Decision Timeline

This should become another permanent reference.

| DecisionWho/SubsystemWhenRuntime information?Can AQE affect it? ||||

| ----- | ----- | ----- | ----- |
----- |

| Parse query/code | SQL engine / API | Planning | No | No |

| Resolve columns | Analyzer | Planning | No | No |

| Logical optimization | Catalyst | Planning | Mostly no | Later runtime adaptations are separate |

| Physical operator selection | Physical planner | Planning | Statistics/config where available | Some decisions can be adapted |

| Initial shuffle partition configuration | SQL/runtime config | Planning/execution setup | Configuration | AQE can coalesce/split |

| Stage boundaries | Scheduler/execution dependencies | Execution preparation | Dependency structure | AQE can create adaptive query stages |

| Task creation | Scheduler | Execution | Runtime state | Runtime scheduling |

| Task placement | Task scheduler / cluster | Execution | Yes | No in the sense of changing scheduling policy |

| Executor allocation | Cluster manager / Spark resource logic | Application/execution | Cluster state | Dynamic allocation can change resources |

| Shuffle read/write | Executors | Execution | Yes | Adaptive stages depend on runtime stats |

| Partition coalescing | AQE | Runtime | Yes | This is AQE itself |

| Skew handling | AQE | Runtime | Yes | Yes |

| Some join changes | AQE | Runtime | Yes | Yes |

The key conceptual division is:

PLANNING TIME

↓

"What should we do?"

EXECUTION TIME

↓

"What resources/data do we actually have?"

AQE

↓

"Given what we now know, should we adapt?"

32. Numerical Simulation A

Let's use your requested:

```
Input = 1 GB
Executors = 4
Cores/executor = 4
```

Assume, purely for illustration:

```
128 MB target input split
```

Then:

```
1 GB / 128 MB
= 8 input partitions
```

So approximately:

```
8 partitions
↓
8 tasks
```

Concurrent capacity:

```
4 executors × 4 cores
= 16 task slots
```

Therefore:

```
8 tasks / 16 slots
≈ 1 wave
```

But this says nothing reliable about actual wall-clock time.

Why?

Because runtime also depends on:

```
storage throughput
CPU complexity
serialization
data format
compression
network
scheduler delay
executor startup
cluster contention
```

This is a **capacity calculation**, not a runtime calculation.

33. Numerical Simulation B

```
100 GB
10 executors
4 cores/executor
```

Capacity:

```
10 × 4
= 40 concurrent task slots
```

If we simplistically assume:

```
128 MB per input partition
```

then:

100 GB / 128 MB
≈ 800 partitions

Therefore approximately:

800 tasks
40 concurrent
≈ 20 waves

Again:

20 waves ≠ execution time

It only gives us a simplified parallelism model.

34. Numerical Simulation C

1 TB
20 executors
8 cores/executor

Task capacity:

20 × 8
= 160 concurrent tasks

At an illustrative:

128 MB

partition sizing:

1 TB / 128 MB
≈ 8,192 partitions

Then:

8,192 / 160
≈ 51.2 waves

So approximately:

52 waves

under the simplified assumptions.

But again:

52 waves
≠
actual runtime

This distinction is non-negotiable for our future calculations.

35. Shuffle Partitions

Input partitions and shuffle partitions are different.

For example:

```
Input
↓
8,000 input partitions
↓
JOIN
↓
Exchange
↓
2,000 shuffle partitions
↓
2,000 downstream tasks
```

The number can change.

Therefore never assume:

```
input partitions
=
shuffle partitions
```

They are controlled by different mechanisms.

In Spark SQL, `spark.sql.shuffle.partitions` controls the number of partitions used for shuffle operations such as joins and aggregations; AQE can subsequently coalesce post-shuffle partitions.

36. Failure Architecture

We will classify failures by architectural location.

```
DRIVER
■
■■■ Driver OOM
■■■ Planning failure
■■■ Collect-too-much-data
■■■ Scheduler / application failure

EXECUTOR
■
■■■ Executor OOM
■■■ GC pressure
■■■ Task failure
■■■ Serialization problems
■■■ Executor lost

SHUFFLE
■
■■■ FetchFailed
■■■ Shuffle explosion
■■■ Huge spill
■■■ Network problems

DATA
■
■■■ Skew
■■■ Small files
■■■ Bad partitioning
■■■ Bad data distribution

CLUSTER
■
■■■ Resource shortage
■■■ Scheduling delay
■■■ Executor allocation
■■■ Infrastructure failure
```

Then every failure gets:

```
WHERE?
↓
WHY?
↓
EVIDENCE?
↓
DIAGNOSTIC ORDER?
↓
FIX?
```

↓
TRADE-OFF?

37. Version Boundaries

This needs special attention because you explicitly requested it.

As of the current Apache Spark release documentation available now, the 4.x line has progressed beyond 4.0; Apache's release index lists Spark **4.2.0**, **4.1.3**, **4.0.4**, and **3.5.9** releases dated July 2026.

So we should **not** build our mental model around "Spark 3.x is Spark."

Instead:

```
GENERAL SPARK CONCEPT
  ↓
VERSION-SPECIFIC IMPLEMENTATION
  ↓
DISTRIBUTION-SPECIFIC BEHAVIOR
```

For our learning, I recommend treating these as separate layers:

Layer A — Stable conceptual Spark

Examples:

```
Driver
Executor
Partition
Task
Stage
Shuffle
Narrow dependency
Wide dependency
Logical plan
Physical plan
```

Layer B — Spark 3.x

Important because much production Databricks material still uses 3.x-era terminology and behavior.

Layer C — Spark 4.x

Spark 4.0 introduced substantial changes, including Scala 2.13 as the default and JDK 17 as the default, along with significant SQL/PySpark changes.

For example, Spark 4.0 changed ANSI SQL mode to be enabled by default.

Layer D — PySpark

Python API behavior and Python/JVM interaction.

Layer E — JVM/Scala internals

Actual Spark engine implementation.

Layer F — Databricks

Databricks runtime, Photon, Delta Lake, platform-specific execution, runtime configurations, etc.

We should **never** silently mix these layers.

38. Spark 3.x vs 4.x — One Important Example

AQE:

```
Spark 3.2+
```

AQE is enabled by default in Apache Spark SQL.

But that does **not** mean:

```
Spark 2.x behavior
=
Spark 3.5 behavior
=
Spark 4.2 behavior
```

So whenever we discuss:

```
AQE
Catalyst
Join selection
Memory
Code generation
PySpark
Spark Connect
```

I will identify the relevant version when the implementation matters.

39. What We Will See in EXPLAIN

Eventually, when you see:

```
== Physical Plan ==
AdaptiveSparkPlan
+- ...
   +- SortMergeJoin
      :- Exchange
      : +- ...
      +- Exchange
      +- ...
```

you should be able to read it as:

```
AdaptiveSparkPlan
↓
Runtime adaptation is possible / active
↓
SortMergeJoin
↓
Both sides require redistribution
↓
Exchange
↓
Shuffle boundary
↓
Downstream tasks will consume shuffle partitions
```

Then you look at UI:

```
How much shuffle?
How balanced?
How much spill?
How long are tasks?
How much GC?
```

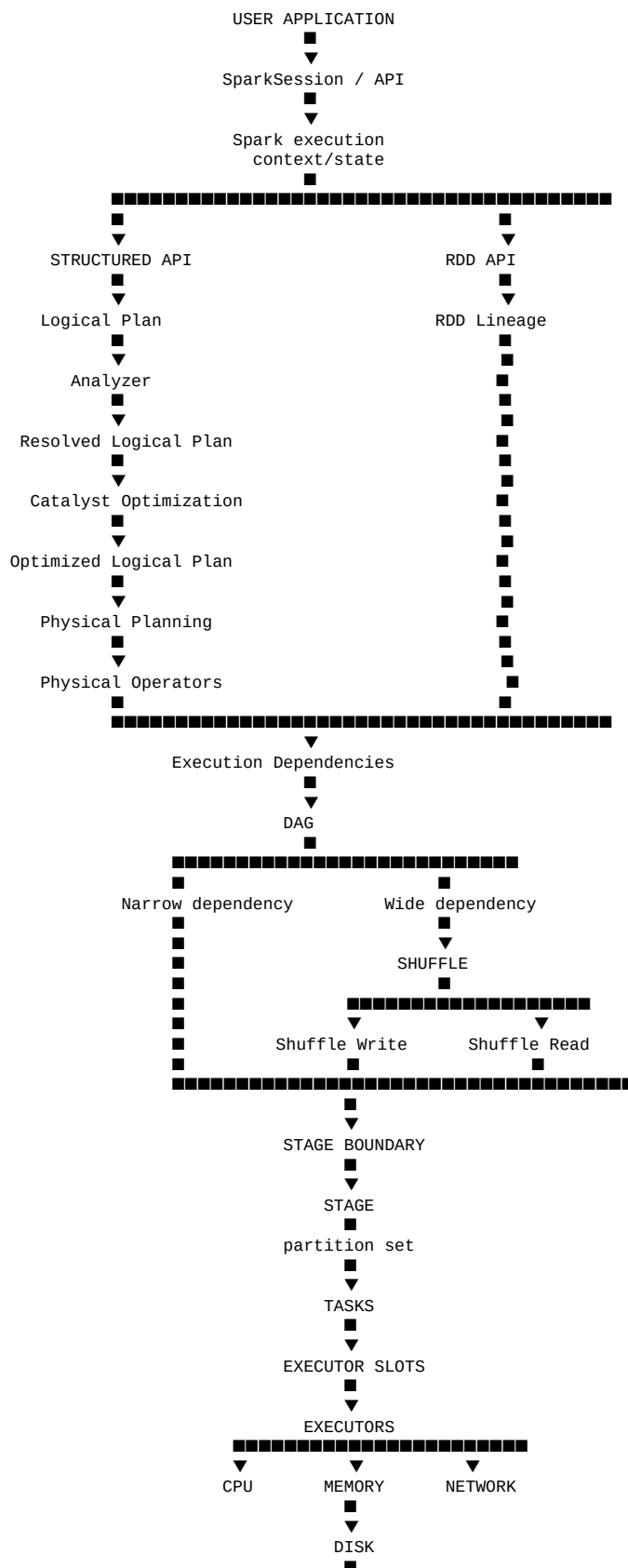
That is the bridge:

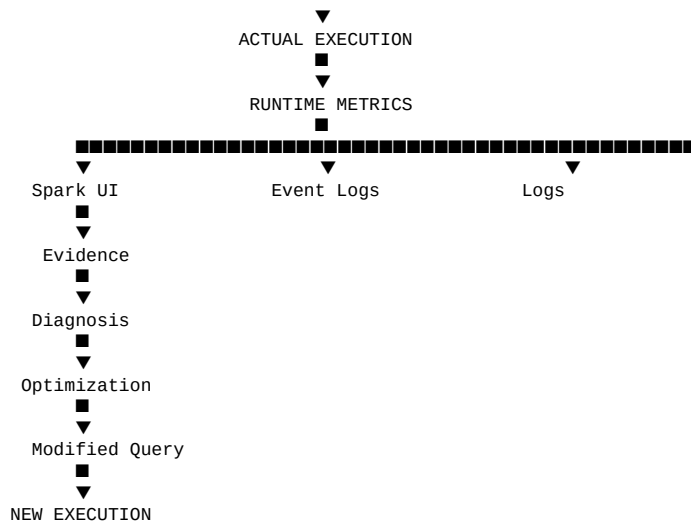
```
EXPLAIN
↓
PREDICTION
↓
```

UI
↓
VALIDATION

40. The Final Connection Map

This is the version I want us to preserve as the ****master reference****:





41. The Entire System in One Sentence

If you remember only one thing:

> **Spark takes a declarative or distributed computation, analyzes and optimizes it, selects physical execution strategies, divides the work around data dependencies and shuffle boundaries, executes partitions as tasks on executors, observes the actual behavior, and gives you enough evidence to diagnose the difference between the intended execution and the real execution.**

That is the mental model underneath everything else.

42. What Spark Decides vs What You Decide

This is another useful separation.

You provide

- Data source
- Transformations
- Joins
- Aggregations
- Filters
- Partitioning hints / explicit repartitioning
- Configuration
- Cluster request

Spark decides

- Logical resolution
- Logical optimizations
- Physical operators
- Some join strategy choices
- Execution dependencies
- Stage structure
- Task scheduling
- Runtime adaptations

Runtime reveals

- Actual data sizes
- Actual partition sizes
- Actual shuffle volume
- Actual skew
- Actual task durations
- Actual spill
- Actual GC
- Actual failures

You diagnose

Why is it slow?
What evidence proves it?
What should change?
What trade-off does that change create?

That final distinction is what separates **using Spark** from **engineering Spark workloads**.

43. Recommended Deep-Dive Sequence

I would modify your proposed order slightly.

Phase 0 — Master mental model

1. Application
2. Driver
3. Executors
4. Cluster manager
5. Data
6. Partitions
7. Tasks
8. Stages
9. Shuffle
10. UI

Why first?

Because otherwise every later concept floats independently.

Phase 1 — Data enters Spark

11. Files
12. Parquet
13. File splits
14. Input partitions
15. Partition pruning
16. Predicate pushdown
17. Column pruning
18. Small files

Why now?

Because you cannot reason about execution without knowing what data Spark actually reads.

Phase 2 — Query representation

19. DataFrame / SQL abstraction
20. Logical plan

- 21. Unresolved plan
- 22. Analyzer
- 23. Resolved plan
- 24. Catalyst
- 25. Optimized logical plan

****Why now?****

Because you need to understand what Spark thinks the query means before understanding how it executes it.

Phase 3 — Physical execution

- 26. Physical planning
- 27. Physical operators
- 28. Exchange
- 29. WholeStageCodegen
- 30. AdaptiveSparkPlan
- 31. Join operators
- 32. Aggregation operators
- 33. Sort

Phase 4 — Distributed execution

- 34. Dependencies
- 35. DAG
- 36. Stage
- 37. Task
- 38. Executor
- 39. Task slots
- 40. Scheduling
- 41. Locality
- 42. Speculation

Phase 5 — Data movement

- 43. Narrow dependency
- 44. Wide dependency
- 45. Shuffle
- 46. Shuffle write
- 47. Shuffle read
- 48. Partitioning
- 49. Repartition
- 50. Coalesce
- 51. Spill

Phase 6 — Performance engineering

- 52. Join strategies
- 53. Broadcast
- 54. Sort-merge
- 55. Shuffled hash
- 56. Skew
- 57. AQE
- 58. Partition sizing
- 59. CPU
- 60. Memory
- 61. Disk
- 62. Network
- 63. Serialization
- 64. GC

Phase 7 — Observability

- 65. Spark UI
- 66. SQL tab
- 67. Jobs
- 68. Stages
- 69. Tasks
- 70. Executors
- 71. Storage
- 72. Event logs
- 73. Metrics

Phase 8 — Diagnosis

- 74. Slow job
- 75. Slow stage
- 76. Straggler
- 77. Skew
- 78. Shuffle explosion
- 79. Spill
- 80. GC
- 81. Executor OOM
- 82. Driver OOM
- 83. FetchFailed
- 84. Executor lost

Phase 9 — Production engineering

- 85. Performance experiments

- 86. Capacity planning
- 87. Cluster sizing
- 88. Reliability
- 89. Idempotency
- 90. Failure recovery
- 91. Production architecture
- 92. Databricks-specific execution
- 93. Delta Lake interaction
- 94. Real-world design cases

This dependency order is stronger than simply following a Spark textbook chapter by chapter.

44. The Learning Loop We Will Use for Every Component

From now on, when we deep-dive something like **Shuffle**, we don't restart Spark.

We locate it:

```
MASTER ARCHITECTURE

Physical Plan
↓
Exchange
↓
Shuffle
↓
Stage Boundary
↓
Tasks
↓
Executors
↓
CPU / Memory / Disk / Network
↓
Spark UI
```

Then we ask:

1. What problem does Shuffle solve?
2. Why does Spark need it?
3. What existed before the shuffle?
4. What input does it receive?
5. Who decides it is necessary?
6. What happens internally?
7. What output does it produce?
8. What happens after it?
9. What CPU cost exists?
10. What memory cost exists?
11. What disk cost exists?
12. What network cost exists?
13. What can go wrong?
14. How does EXPLAIN show it?
15. How does Spark UI show it?
16. What metrics prove the behavior?
17. What changes by Spark version?
18. What numerical example demonstrates it?
19. What controlled experiment proves it?
20. Where does it return to the master map?

That is essentially the 20-question framework you requested, and I recommend keeping it unchanged as our **deep-dive contract**.

45. The Skill We Are Ultimately Building

The final objective is not:

"I know what a shuffle is."

It is:

I see:

```
df.join(...)
  .groupBy(...)
  .agg(...)
```

↓

I predict:

Exchange

↓

Shuffle

↓

Join

↓

Aggregation

↓

I inspect:

EXPLAIN FORMATTED

↓

I predict:

Stage boundaries

Task counts

Potential shuffle volume

↓

I run it

↓

Spark UI shows:

one huge task

+

massive shuffle read

+

spill

↓

I infer:

partition imbalance / skew

↓

I trace:

Task

→ Partition

→ Shuffle

→ Join

→ Join key

→ Source code

↓

I test a hypothesis

↓

I change the design

↓

I understand the trade-off

↓

I verify the new execution.

That is the production-level mental model you described in the uploaded specification: reconstructing **what Spark decided, why it decided it, what happened internally, what the cluster actually did, where the bottleneck is, how to prove it, and what trade-off a fix creates**.

Architecture status

I would consider the **structure** of this Version 1 stable enough to review, but I would **not yet freeze it as the final PDF architecture**.

The main things I want you to challenge are:

1. **Driver → SparkSession → SparkContext → Scheduler relationship**
2. **Logical Plan → Catalyst → Physical Plan**
3. **Physical Plan → execution representation → DAG**
4. **DAG → shuffle boundaries → stages**
5. **Stage → partitions → tasks**
6. **Input partitions vs shuffle partitions**
7. **Storage partitions vs Spark partitions**
8. **Executor vs executor core/task slot**
9. **Initial physical planning vs AQE**
10. **Planned execution vs actual Spark UI evidence**
11. **RDD branch vs DataFrame/SQL branch**
12. **Where JVM/PySpark boundaries actually occur**

Those are the places where a superficially correct Spark diagram can produce a **wrong mental model**.

I will use this architecture as the stable reference rather than silently changing the structure when we later deep-dive individual components, exactly as specified in your uploaded master prompt.